

# Понятие индекса, кластерные и некластерные индексы

**Индекс** — объект базы данных, создаваемый с целью повышения производительности поиска данных. Таблицы в базе данных могут иметь большое количество строк, которые хранятся в произвольном порядке, и их поиск по заданному критерию путем последовательного просмотра таблицы строка за строкой может занимать много времени.

Индекс формируется из значений одного или нескольких столбцов таблицы и указателей на соответствующие строки таблицы и, таким образом, позволяет искать строки, удовлетворяющие критерию поиска. Ускорение работы с использованием индексов достигается в первую очередь за счёт того, что индекс имеет структуру, оптимизированную под поиск - например, сбалансированного дерева.

## Типы индексов

Индексы могут быть уникальными (UNIQUE INDEX) и не уникальными (INDEX). Уникальные индексы гарантируют, что в таблице не будет повторяющихся значения по указанным в нем полям. Это позволяет контролировать целостность данных и посредством уникальных индексов накладывать ограничения собственной бизнес-логики. Так же уникальные индексы позволяют ASE эффективно реализовывать механизм блокировок, снимая количество блокировок и нагрузку по их обработке во время операций вставок и изменения записей в таблице. Исходя из этого рекомендуется на каждую таблицу иметь хотя бы один уникальный индекс. Как уникальные, так и не уникальные индексы могут включать в себя поля, которым разрешено содержать пустое значение (NULL).

ASE автоматически создает уникальные индексы для первичных ключей (PRIMARY KEY) и ограничений уникальности (CONSTRAINT UNIQUE). Так же ASE автоматически создает индексы на ограничения связи между таблицами (FOREIGN KEY), которые могут являться не уникальными. После создания уникального индекса, при попытке вставить повторяющееся значение, вы получите сообщение об ошибке.

## Кластерные и некластерные индексы

Существует два типа индекса: **кластерный** (clustered) и **некластерный** (nonclustered).

При наличии кластерного индекса строки таблицы упорядочены по значению ключа этого индекса, т.е. кластерный индекс предполагает, что при его использовании физический порядок размещения строк в таблице соответствует их индексному порядку. Для таблицы может быть построен только один кластерный индекс и произвольное число некластерных (249). Кластерный индекс в некоторых СУБД может быть уникальным индексом - это означает что все ключи записей должны быть уникальные. Если существуют записи с одинаковыми значениями, СУБД делает их уникальными, добавляя номера из внутреннего (невидимого снаружи) счетчика. Но добавляемое значение не будет видно ни конечному пользователю, ни разработчику. Оно просто имеет внутреннюю поддержку для идентификации записи.

Кластерный индекс — это древовидная структура данных, при которой значения индекса хранятся вместе с данными, им соответствующими. И индексы, и данные при такой организации упорядочены. При добавлении новой строки в таблицу, она дописывается в нужную ветку древовидной структуры, соответствующую ей по сортировке.

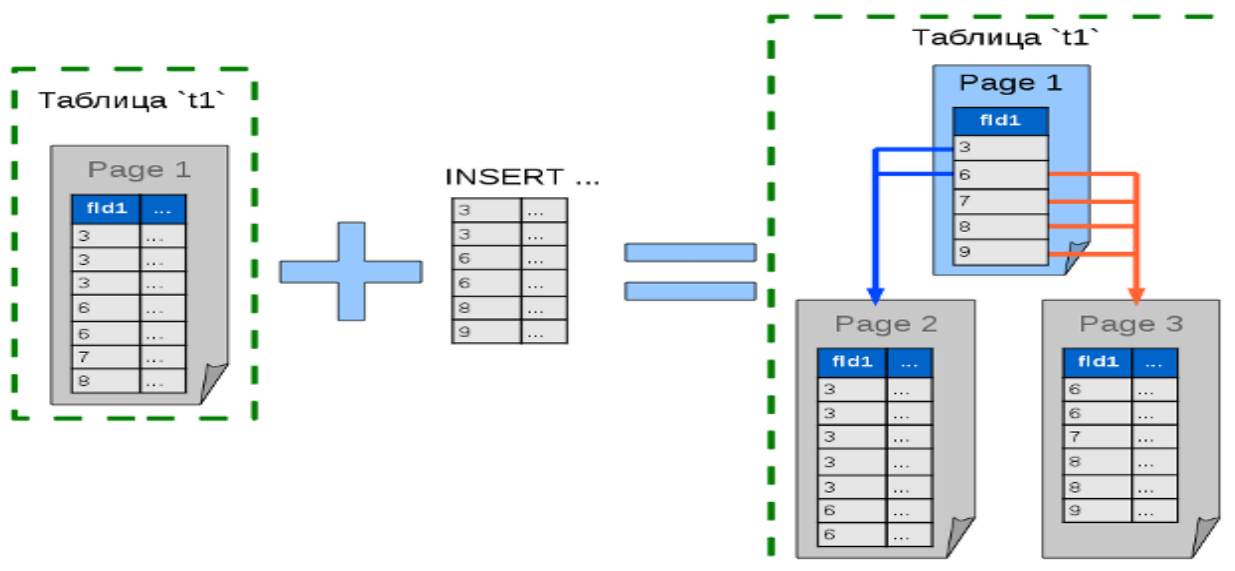
Данные в базах данных хранятся страницами predetermined размера. Размер одной страницы — это предельный размер узла древовидной структуры, от которого зависит в какой момент начнётся ветвление. Если вся таблица помещается в одну страницу, то она хранится в виде плоского списка, отсортированного по ключевому полю, без отдельной индексной таблицы:

16 Кб  
страница InnoDB

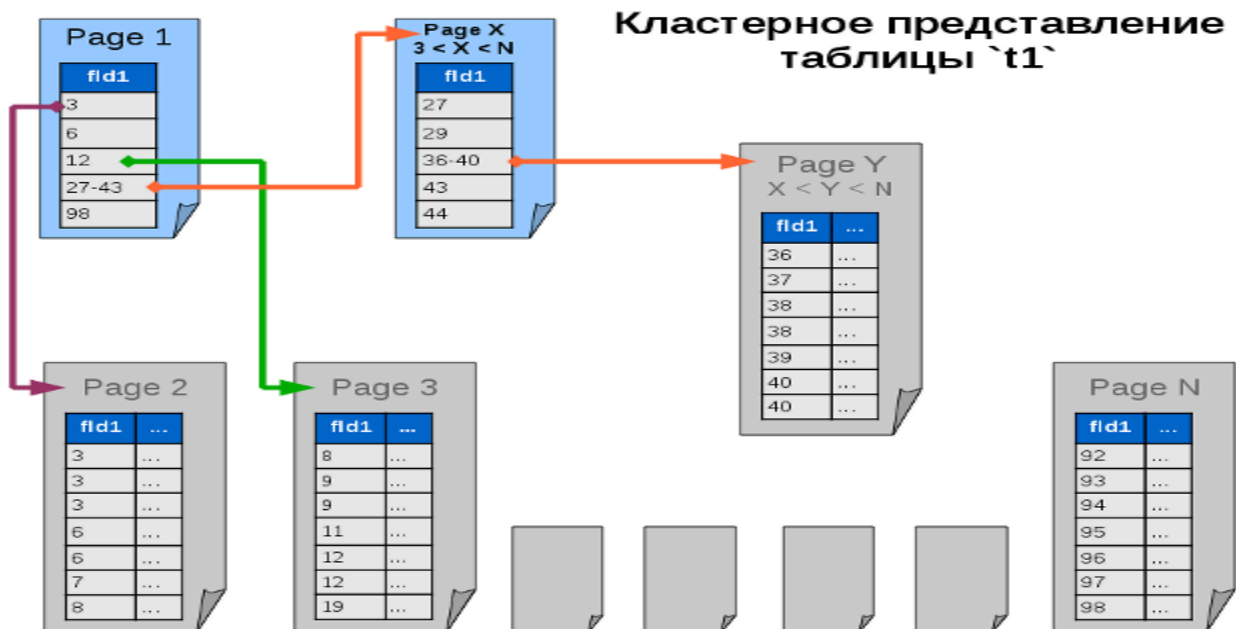
Таблица `t1`

| fld1 | ... | fldN |
|------|-----|------|
| 3    | ... | ...  |
| 3    | ... | ...  |
| 3    | ... | ...  |
| 6    | ... | ...  |
| 12   | ... | ...  |
| 12   | ... | ...  |
| 27   | ... | ...  |

Когда данные перестают помещаться в одну страницу, список превращается в дерево. Страница с данными разделяется на две, причём в том узле (на той странице), где раньше были данные, теперь располагается индекс, охватывающий обе новые страницы. Конкретный узел такого дерева обязан включать в себя индексы всех дочерних узлов или конечные данные, если узел последний. Узлы могут ссылаться друг на друга только в одном направлении: от родителя к потомку:



По мере добавления всё новых и новых данных, дерево будет усложняться и углубляться. И чем больше оно будет и ветвистее, тем больший выигрыш даст такая схема хранения данных:



#### Примечание к рисунку:

**Серые страницы** идентичны странице первого этапа — это просто отсортированные данные, **листья** (конечные узлы) нашего дерева. **Голубые страницы** — это промежуточные узлы дерева, содержащие только индекс и не содержащие данных. **Стрелками** помечены пути поиска определённых значений ключа.

Элементы индекса хранятся в виде строк на страницах индекса. Их формат похож на формат строк данных на страницах данных. Элементы индекса хранят значения ключа и указатели на нижние уровни индекса, на страницы данных или на отдельные строки данных.

Индексы Adaptive Server имеют структуру B-дерева, поэтому каждый узел в структуре индекса может иметь несколько потомков.

Элементы индекса обычно намного меньше, чем строка данных на странице данных, а страницы индекса заполнены гораздо плотнее, чем страницы данных. Если строка данных занимает 200 байтов (включая служебную часть строки), на каждой странице находится 10 строк.

Индекс по 15-байтовому столбцу содержит около 100 строк для каждой страницы индекса (для указателя на каждую строку требуется от 4 до 9 байтов в зависимости от типа и уровня индекса).

Индексы могут иметь несколько уровней:

- корневой уровень;
- листовой уровень;
- промежуточный уровень.

**Корневой уровень** — самый высокий уровень индекса. Существует только одна корневая

страница. Если таблица с блокировкой всех страниц настолько мала, что весь индекс уместается на одной странице, то промежуточный и листовой уровни отсутствуют, и указатели на страницы данных хранятся на корневой странице.

Для таблиц с блокировкой только данных всегда существует листовой уровень между корневой страницей и страницами данных.

Для более крупных таблиц корневая страница хранит указатели на страницы индекса промежуточного уровня или на страницы листового уровня.

Самый нижний уровень индекса – **листовой**. На этом уровне индекс содержит значение ключа для каждой строки в таблице, причем эти строки упорядочены по индексному ключу.

- Для кластерных индексов по таблицам с блокировкой всех страниц листовой уровень – это сами данные. Никакой другой уровень индекса не содержит по одной строке индекса на каждую строку данных.
- Для некластерных индексов и кластерных индексов по таблицам с блокировкой только данных листовой уровень содержит значение индексного ключа для каждой строки, указатель на страницу, где хранится строка, и указатель на строки на этой странице данных.

Листовой уровень расположен непосредственно “над” данными. Он содержит одну строку индекса для каждой строки данных. Строки индекса на странице индекса хранятся в порядке значений ключа.

Все уровни между корневым и листовым – это **промежуточные уровни**.

Индекс по крупной таблице или индекс с длинными ключами может иметь много промежуточных уровней. Очень маленькая таблица с блокировкой всех страниц может вообще не иметь промежуточного уровня: корневые страницы указывают прямо на листовой уровень.

Примерно это будет выглядеть примерно так:

```
-- Корневые страницы уровня 1
  -- Страницы уровня уровня 2
    -- Страницы уровня уровня 3
      -- Страницы листа (ссылки на физические записи)
```

В итоге, чтобы найти нужную запись, СУБД будет вынуждена просмотреть и найти значение в дереве уровня 1, далее спуститься и обработать уровень 2, затем сделать то же самое с уровнем 3 и только потом получить ссылку на физическую запись в таблице. В итоге, можно сказать, чем меньше уровней страниц в индексе, тем быстрее он считывается и большее кол-во страниц оседает в кэше для повторного использования. На кол-во уровней в индексе влияет размер страницы, длина используемых полей и разветвленность индекса.

Соответственно, индекс, указывающий на другую страницу, не обязательно ведёт сразу на

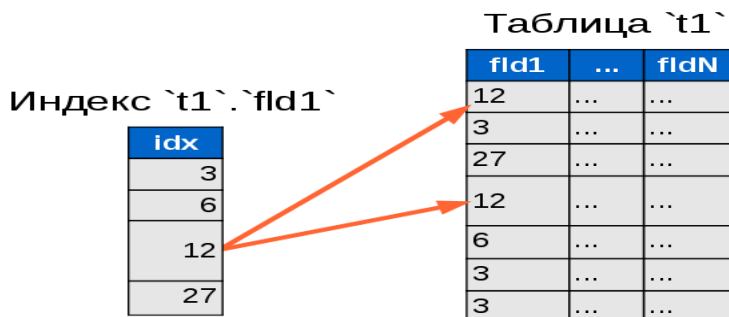
страницу с данными. Индекс может указывать на страницу с промежуточным индексом. Возможно, при больших объемах таблицы, БД придется провести больше итераций поиска, но каждая такая итерация включает минимальный объем данных, а потому в целом всё равно поиск проходит быстрее. Здесь действует простое правило, актуальное для любого типа индекса: чем разнообразнее данные, тем эффективнее использовать индекс для поиска конкретных значений.

Поскольку данные являются частью индекса, отсортированы и целенаправленно фрагментированы, очевидно что для одной таблицы может использоваться только один кластерный ключ. Из такой сложной логики хранения индексов и данных есть ещё одно важное следствие: операции записи, а особенно изменение имеющихся данных ключевых полей - крайне ресурсоёмкий процесс. Старайтесь использовать для кластерных индексов редко изменяемые поля.

Что касается сложных (составных) кластерных ключей, для них действует абсолютно такая же схема, только сортировка данных осуществляется по нескольким полям.

Если в таблице нет кластерного индекса, таблица называется кучей.

**Некластерный** индекс, созданный для такой таблицы, содержит только указатели на записи таблицы в древовидной структуре, листья которой упорядочены по ключевому полю этого индекса:



Рассмотрим некластерные индексы подробнее. Упрощённо некластерный индекс можно представить как отдельную таблицу, каждая строка в которой ссылается на одну или несколько строк в таблице с данными. Строки в индексной таблице упорядочены и сгруппированы по значениям ключевых полей. Представим запрос:

```
SELECT * FROM `t1` WHERE `fld1` = 12;
```

Совсем без индексации будет прочитана и проверена каждая строка, и неудовлетворяющие условию строки просто не попадут в результат. Но прочитаны они будут.

При использовании некластерного индекса, задача поиска сильно ускоряется.

**Во-первых**, индексная таблица весит много меньше таблицы с данными, а значит элементарно может быть прочитана быстрее.

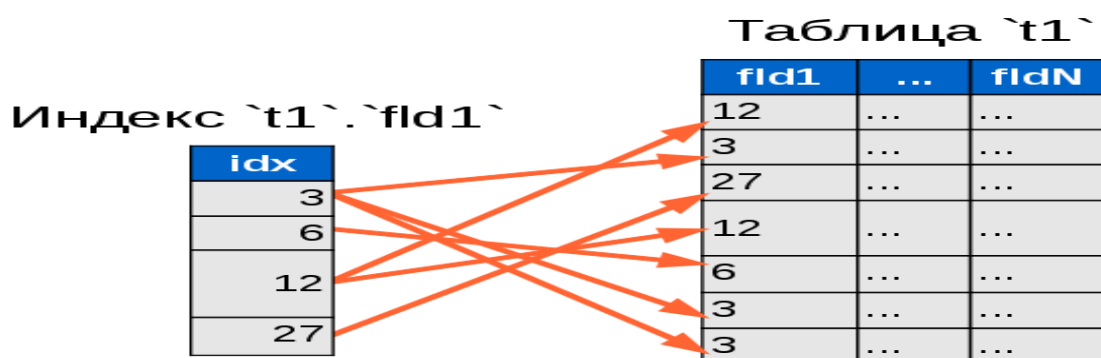
**Во-вторых**, СУБД чаще всего стараются кешировать индексы в оперативную память, которая сама по себе производительней жёсткого диска.

**В-третьих**, в индексах отсутствуют дублирующиеся строки. А значит, как только мы нашли первое значение, поиск можно прекращать — оно же и последнее.

**В-четвёртых**, данные в индексе отсортированы, что вместе с предыдущим пунктом позволяет использовать алгоритм бинарного поиска (он же метод деления пополам), эффективность которого многократно превосходит простой перебор.

Если ресурсы позволяют, таблицу данных тоже можно кешировать в оперативную память. Однако индексам и месту для них в оперативной памяти, по понятным причинам, принято уделять больше внимания.

Если представить все указатели индексной таблицы на строки в таблице данных, получится достаточно сложная «паутина»:



Множество пересекающихся стрелок, подводит нас к проблеме, которую создаёт некластерный индекс - **фрагментация**.

### Фрагментация индексов

Внутри файлов данных, в структурах, хранящих данные таблиц и индексов может произойти два базовых типа фрагментации:

- Фрагментация внутри отдельных страниц данных и индекса (именуемая внутренней фрагментацией)
- Фрагментация внутри структур индекса или таблиц, состоящих из страниц (именуемая логической фрагментацией просмотра или фрагментацией просмотра по экстендам)

**Внутренняя фрагментация** — это наличие в странице больших пустых пространств. Пустое пространство может возникнуть, если каждая таблица или запись в индексе превышают по

размерам половину страницы, поскольку тогда на странице можно сохранить лишь одну запись. Исправить это очень сложно или невозможно, поскольку для исправления необходимо изменение схемы таблицы или индекса, например, путем изменения ключа индекса на что-то, что не вызывает случайные точки вставки.

Чаще внутренняя фрагментация вызывается изменениями данных, такими как вставки, обновления и удаления, что может оставить на странице пустые места. В зависимости от схемы таблицы/индекса и характеристик приложения это пустое место может оказаться непригодным для повторного использования после его появления, что может привести к постоянному росту неиспользуемого места в базе данных.

Неиспользуемое место на страницах данных/индекса может, таким образом, привести к хранению того же объема данных на большем числе страниц. Это не только приводит к большему расходу места на диске, но и значит, что запросу надо провести больше операций ввода/вывода, чтобы прочесть тот же объем данных. И все эти дополнительные страницы занимают больше места в кэше данных, занимая тем самым память сервера.

**Логическая фрагментация** просмотра вызывается операцией, именуемой разбиением страницы. Это происходит, когда запись необходимо вставить на определенную страницу индекса (согласно определению ключа индекса), но на странице недостаточно места, чтобы разместить вставляемые данные. Страница разбивается пополам, и примерно 50% записей перемещаются на свежевыделенную страницу. Эта новая страница обычно не является физически смежной со старой и, следовательно, именуется фрагментированной. Концепция фрагментации просмотра по блокам аналогична. Фрагментация внутри структур таблиц/индексов влияет на возможность СУБД выполнять эффективные просмотры как по всей таблице/индексу, так и ограниченные предложением WHERE запроса (такие как `SELECT * FROM MyTable WHERE Column1 > 100 AND Column1 < 4000`).

Оптимизатор СУБД может принять решение вообще не использовать индексы для поиска по небольшим таблицам (до пары десятков записей — зависит от конкретной структуры данных и индекса), потому что поиск простым перебором читает данные последовательно. А указатель в индексе ссылается на разрозненные участки данных. И прыжки по ссылкам из индекса в конечном итоге могут стоить дороже полного перебора.

## Составные индексы

Индекс может быть создан на основании нескольких полей. В этом случае очень важен порядок следования полей в индексе. Составной индекс следует использовать в том случае, когда нужно вести поиск по комбинации значений в нескольких столбцах. В этом случае комбинацию значений данных можно рассматривать как одно составное значение. Дело в том, что получить набор данных по запросу, затрагивающему только первый из проиндексированных столбцов, можно. Однако в большинстве СУБД невозможно или неэффективно получение данных только по второму и далее проиндексированным столбцам (без ограничений на первый столбец). Например, представим себе телефонный справочник, отсортированный вначале по городу, затем по фамилии, и затем по имени. Если вы знаете город, вы можете легко найти все телефоны этого

города. Однако в таком справочнике будет весьма трудоёмко найти все телефоны, записанные на определённую фамилию — для этого необходимо посмотреть в секцию каждого города и поискать там нужную фамилию.

Когда используются составные индексы, состоящие из множества полей, они имеют такую характеристику, как разветвленность. Чем больше значений имеет поле индекса, тем больше веток имеет индекс, а значит он занимает больше страниц в базе данных, требует большего времени на чтение и меньше кэшируется.



# Создание и уничтожение индекса

Упрощенный синтаксис:

```
CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED] INDEX index_name
      ON [[database.]owner.]table_name( column_name [asc | desc]
                                         [, column_name]... )
```

|                    |                                                |
|--------------------|------------------------------------------------|
| <b>index_name</b>  | название создаваемого индекса                  |
| <b>database</b>    | база , в которой создается индекс              |
| <b>owner</b>       | владелец                                       |
| <b>table_name</b>  | название индексируемой таблицы                 |
| <b>column_name</b> | имя столбца, по которому индексируется таблица |

**Пример:**

```
CREATE INDEX xxx_xxx_name ON xxx(xxx_name)

CREATE UNIQUE NONCLUSTERED INDEX xxx_parent_coulumn0
ON xxx( column0 desc, xxx_parent )
```

Название индекса должно удовлетворять правилам создания имен для идентификаторов. Столбец и имя таблицы определяет столбец, который Вы хотите индексировать и таблицу, которая содержит этот столбец. Индексное выражение не может содержать столбец типа bit, image или text.

Команда **DROP INDEX** удаляет информацию об индексе из базы данных и освобождает выделенные под него дисковые ресурсы

Синтаксис:

```
DROP INDEX table_name.index_name [, table_name.index_name ...]
```

**Пример:**

```
DROP INDEX xxx.xxx_xxx_name

DROP INDEX xxx.xxx_parent_coulumn0
```

Для того чтобы изменить индекс, его нужно удалить и создать заново.

## Получение информации об индексах

Команда **sp\_helpindex** возвращает информации о существующих индексах.

`sp_helpindex objname`, где `objname` – имя таблицы в текущей базе данных.

**sp\_statistics** возвращает информацию об индексах в табличной форме.

```
sp_statistics table_name [, table_owner]
               [, table_qualifier] [, index_name] [, is_unique]
```

|                        |                                                                                     |
|------------------------|-------------------------------------------------------------------------------------|
| <b>table_name</b>      | имя таблицы. Допускает групповые символы                                            |
| <b>table_owner</b>     | владелец таблицы. Допускает групповые символы                                       |
| <b>table_qualifier</b> | имя базы данных. Допустимые значения - имя текущей базы данных или NULL             |
| <b>is_unique</b>       | при <code>is_unique = «Y»</code> выводится информация только об уникальных индексах |

**sp\_estspace** дает оценку дискового пространства, требуемого для таблицы и индексов, и времени, необходимого для построения индекса, а **sp\_spaceused** с параметром «1» показывает количество реально занимаемого пространства.

# Использование команды update statistics

Эта команда обновляет информацию о распределении ключевых значений в указанных индексах или в указанных столбцах, для всех столбцов в индексе или для всех столбцов в таблице.

Синтаксис:

```
update statistics имя_таблицы
    [ [имя_индекса] | [ ( список_столбцов ) ] ]
    [using шаг values]
    [with consumers = потребители ]

update index statistics имя_таблицы [имя_индекса]
    [using шаг values]
    [with consumers = потребители ]
```

**имя\_таблицы** - имя таблицы, с которой связан индекс. Имя таблицы необходимо, поскольку Transact-SQL не требует, чтобы имена индексов в базе данных были уникальны.

**имя\_индекса** - имя обновляемого индекса. Если имя индекса не задано, в указанной таблице обновляется статистика распределения для всех индексов.

**список\_столбцов** - список столбцов, разделенных запятыми.

**using шаг values** - указывает количество шагов гистограммы. Для столбцов без статистики значение по умолчанию – 20. Если статистика столбца уже существует в таблице sysstatistics, значение по умолчанию – текущее количество шагов.

**[with consumers = потребители]** - указывает количество процессов-потребителей, используемых для сортировки, если указан список\_столбцов и включена параллельная обработка запросов.

**index** - указывает, что статистика обновляется для всех столбцов в индексе.

## Примеры:

Формирование статистики для столбца **price** таблицы **titles**:

```
update statistics titles (price) using 40 values
```

Формирование статистики для всех столбцов во всех индексах таблицы **authors**:

```
update index statistics authors
```

Формирование статистики для всех столбцов в индексе **au\_names\_ix** таблицы **authors**:

```
update index statistics authors au_names_ix
```

## Использование

- Сервер Adaptive Server хранит статистику о распределении ключевых значений в каждом индексе и использует эти статистические данные при выборе индексов для обработки запроса.
- При создании некластерного индекса для таблицы, содержащей данные, команда **update statistics** автоматически запускается для нового индекса. При создании кластерного индекса для таблицы, содержащей данные, команда **update statistics** автоматически запускается для всех индексов.
- Оптимизация запросов зависит от точности статистики. Если произошли значительные изменения ключевых значений индекса, то следует перезапустить команду **update statistics** для этого индекса или столбца.
- Команда **update statistics** используется, если значительная часть данных в индексированном столбце была изменена, удалена или добавлена в этот столбец (то есть если есть основания полагать, что распределение ключевых значений изменилось).
- Команда **update statistics** с именем таблицы и именем индекса обновляет статистику для начального столбца индекса. Если команда **update statistics** используется только с именем таблицы, она обновляет статистику для начальных столбцов всех индексов таблицы.
- Команда **update index statistics** с именем таблицы и именем индекса обновляет статистику для всех столбцов в указанном индексе. Если команда **update index statistics** используется только с именем таблицы, она обновляет статистику для всех столбцов во всех индексах таблицы.
- Указание имени неиндексированного или не первого столбца индекса формирует статистику для этого столбца без создания индекса.
- Указание более одного столбца в списке столбцов формирует или обновляет гистограмму для первого столбца и статистику плотности для всех префиксных подмножеств списка столбцов.
- При использовании команды **update statistics** для формирования статистики по столбцу или списку столбцов эта команда должна просмотреть таблицу и выполнить сортировку.
- Инструкция **with consumers** предназначена для применения к секционированным таблицам, хранимым на RAID-устройствах, которые воспринимаются сервером Adaptive Server как отдельные устройства ввода-вывода, однако могут обеспечивать высокую пропускную способность, необходимую для параллельной сортировки.
- Команда **update index statistics** формирует последовательность операций обновления статистики, использующих те же блокировки, просмотры и сортировки, что и эквивалентные команды на уровне индекса и столбца. Например, если у таблицы

salesdetail есть некластерный индекс sales\_det\_ix по столбцу salesdetail(stor\_id, ord\_num, title\_id), команда

```
update index statistics salesdetail
```

выполняет следующие операции update statistics:

```
update statistics salesdetail sales_det_ix
update statistics salesdetail (ord_num)
update statistics salesdetail (title_id)
```

- Команда **update all statistics** формирует последовательность операций **update statistics** для каждого индекса таблицы, за которой идет последовательность операций **update statistics** для всех неиндексированных столбцов, а затем операция **update partition statistics**.
- Команда **update statistics** не выполняется для системных таблиц базы данных master во время перехода на более новую версию. Индексы существуют по столбцам, запросы к которым выполняют большинство системных процедур, потому выполнение команды **update statistics** для этих таблиц обычно не требуется. Однако выполнение команды **update statistics** разрешено в отношении всех системных таблиц во всех базах данных, за исключением следующих нестандартных таблиц: syscurconfigs, sysengines, sysgams, syslisteners, syslocks, syslogs, syslogshold, sysmonitors, sysprocesses, syssecmechs, systestlog и systransactions. При выполнении запросов к этим таблицам они строятся из внутренних структур.

## Общие рекомендации по использованию индексов

### 1. **Создавайте индекс по столбцам, которые часто используются в выражениях WHERE, ORDER BY, GROUP BY.**

Эти столбцы идеальные кандидаты для создания индексов. Вам следует очень внимательно проанализировать ваши запросы, чтобы избежать создания бесполезных индексов.

### 2. **Ваши индексы должны быть как можно меньше.**

Из-за того, что каждый индекс занимает дисковое пространство, попробуйте минимизировать размер ключа индекса, чтобы избежать чрезмерного размера базы. Таким образом уменьшится количество операций чтения, необходимых для чтения индекса и повысит общую производительность индекса.

### 3. **Удаляйте индексы, которые не используются.**

Т.к. каждый индекс занимает дисковое пространство и замедляет вставку, удаление и обновление строк в таблицах, вам следует удалять неиспользуемые индексы.

### 4. **Постарайтесь создать индексы по столбцам, которые имеют целые, а не символьные значения.**

Потому что целые значения, как правило, меньшего размера, чем символьные (размер типа INT – 4 байта, размер типа BIGINT – 8 байт), вы можете уменьшить количество страниц индекса, которые используются для хранения ключей индекса. Это уменьшит количество чтений, требуемых для чтения индекса и увеличит общую производительность.

### 5. **Ограничьте количество индексов, если ваше приложение часто обновляет данные.**

Т.к. каждый индекс занимает дисковое пространство и замедляет добавление, удаление и обновление строк, поэтому вы должны создавать новые индексы только после анализа использования данных, типа и частоты выполняемых запросов, и того, как ваши запросы будут использовать новые индексы. В большинстве случаев преимущества от создания новых индексов перевешивают такие недостатки, как дополнительное используемое пространство и медленные модификации данных. Однако, избегайте использования избыточных индексов, создавайте их, только когда это необходимо. Для таблиц, которые используются исключительно для чтения, число индексов может быть увеличено.

### 6. **Убедитесь, что индекс, который вы пытаетесь создать, еще не существует.**

Имейте в виду, что при создании первичного ключа или уникального ключа СУБД автоматически создает индекс на столбцы, которые участвуют в вашем ключе. Если указать другое имя индекса, то можно создавать индексы для того же столбца снова и снова.

**7. Создавайте кластерный индекс вместо некластерного для увеличения производительности запросов, которые возвращают диапазон значений, и для запросов, содержащих GROUP BY или ORDER BY выражения и возвращающих отсортированные результаты.**

Т.к. каждая таблица может содержать только один кластерный индекс, поэтому вам следует очень тщательно выбирать столбцы для индекса. Постарайтесь проанализировать все ваши запросы, выберите наиболее часто используемые и включите в кластерный индекс только те столбцы, которые обеспечивают наибольшее преимущество в производительности.

**8. Создавайте некластерные индексы для увеличения производительности запросов, которые возвращают несколько строк и там где индексы имеют хорошую избирательность (selectivity).**

Таблицы могут содержать до 249 некластерных индексов, в отличие от кластерного индекса, который может быть только один для каждой таблицы. Однако, вам следует создавать некластерные индексы очень тщательно, как и в случае кластерными индексами, потому что каждый индекс занимает дисковое пространство и влияет на скорость модификации данных.

**9. Создавайте кластерный индекс для столбцов, которые не обновляются очень часто.**

Т.к. листья в дереве некластеризованного индекса содержат ключ кластерного индекса, поэтому если у таблицы создан кластерный индекс, то каждый раз когда обновляются столбцы входящие в него – все некластерные индексы тоже будут обновляться.

**10. Создавайте кластерный индекс на основе одного столбца или как можно меньшего их количества.**

Т.к. некластерные индексы содержат ключ кластерного индекса в листьях дерева и некластерные индексы используют кластерный для поиска, то создание кластерного индекса по одному столбцу уменьшит размер не только кластерного индекса, но и всех некластерных.

**11. Следует избегать создания кластерного индекса на столбцах с автоинкрементом.**

Например, если у таблицы автоинкрементальный первичный ключ и создан кластерный индекс по этому столбцу, то каждый раз когда происходит вставка в эту таблицу, строки будут добавляться в конец таблицы. А когда будет вставляться много строк, то может произойти много запросов, которые пытаются прочитать и записать данные в одну область в одно время. Это происходит из-за того, что устройства ввода-вывода являются узким местом. Заметьте, что по умолчанию ASE создает кластерный индекс по столбцам, которые входят в первичный ключ. В этом случае вам следует явно указать, что вы хотите создать некластерный индекс для этих столбцов.

**12. Создавайте кластерный индекс для каждой таблицы.**

Если вы создаете таблицу без кластерного индекса, то данные будут храниться неупорядоченно. Такая структура называется куча. Для улучшения работы параллельных запросов, следует создавать кластерные индексы для каждой таблицы.

**13. Не создавайте индексы по столбцам с низкой избирательностью (selectivity).**

Например, не создавайте индекс по столбцам, в которых содержится много дублирующихся данных, например для столбца “Пол”, который может содержать только значения “Мужской” и “Женский”, т.к. в этом случае недостатки от дополнительного занятого дискового пространства и медленной модификации перевешивают преимущества в скорости поиска по такому индексу.

**14. Если вы создаете композитный индекс, то постарайтесь располагать ключи индекса в порядке уменьшения избирательности (selectivity), т.е. ключ с наибольшей избирательностью должен быть самым левым.**

Порядок столбцов в композитных индексах очень важен, он может увеличить шанс использования индекса в запросах.

**15. Если вы создаете композитный индекс, то располагайте в начале ключи индекса, которые часто используются в WHERE выражениях.**

Порядок столбцов в композитных индексах очень важен. Индекс будет использоваться в запросе, только если столбцы, которые находятся в WHERE, являются самыми левыми ключами индекса. Например, если вы создаете композитный индекс “Name, Age”, то при поиске по Name индекс будет использован, а при поиске по Age, он использоваться не будет.

**16. Создавайте индекс для столбцов, которые часто используются в JOIN’ах.**

Это может существенно увеличить производительность запросов с JOIN’ами по этим столбцам.

**17. Подумайте над созданием целочисленного первичного суррогатного ключа.**

Каждая таблица должна содержать первичный ключ (уникальный идентификатор строки в таблице). Суррогатный первичный ключ это поле, которое имеет уникальное значение, но не относится никак к самой записи, и пользователи не должны никогда видеть или менять такой первичный ключ. Некоторые разработчики используют суррогатные первичные ключи, другие используют данные самих полей, как первичный ключ. Если первичный ключ содержит много полей и имеет большой размер – задумайтесь о создании суррогатного ключа. Это может увеличить производительность ваших запросов.

**18. Рассмотрите возможность создания индексов по всем столбцам, которые используются наиболее часто в выражении WHERE, которое содержит операторы ИЛИ.**

Если в выражении WHERE содержится запрос, который содержит оператор ИЛИ и если из них ни один столбец не будет покрыт индексом, то запросу придется полностью просканировать таблицу или кластерный индекс. Поэтому в таких случаях, создавайте индексы на все такие столбцы, что существенно увеличит производительность ваших запросов.

**19. Если ваше приложение выполняет запросы над таблицей очень часто, то рассмотрите вопрос о создании покрывающего индекса, включающего все столбцы, требуемые в запросе.**

Покрывающий индекс, это индекс, который включает все столбцы из запроса. Создание таких индексов может увеличить производительность, т.к. все данные для запросов будут уже содержаться в индексе, и не придется получать их из таблицы. Покрывающие индексы могут



существенно увеличить производительность запросов, так как они экономят на большом количестве операций ввода/вывода.

**20. Периодически перестраивайте все индексы во всех таблицах для уменьшения фрагментации.**

Фрагментированные данные могут заставить СУБД производить дополнительные ненужные чтения, поэтому запросы на сильно фрагментированных таблицах будут выполняться очень плохо. Так что стоит периодически перестраивать индексы для уменьшения фрагментации.

Несмотря на достоинства, индексы так же имеют и ряд недостатков. Первый из них – индексы занимают дополнительное место на диске и в оперативной памяти. Каждый раз когда вы создаете индекс, вы сохраняете ключи в порядке убывания или возрастания, которые могут иметь многоуровневую структуру. И чем больше/длиннее ключ, тем больше размер индекса. Вторым недостатком – замедляются операции вставки, обновления и удаления записей. Однако алгоритмы построения индексов разработаны таким образом что бы иметь как можно меньший негативный эффект для указанных операций и даже позволять выполнять их быстрее.

## Представления

**Представление (view)** – это объект базы данных, представляющий собой именованный оператор SELECT, сохраненный в системном словаре базе данных. С помощью представлений можно создавать логические подмножества или комбинации данных различных таблиц. Хотя представления часто называют виртуальными таблицами, никаких данных они не содержат. Данные ассоциируются с представлением в момент обращения к нему пользователя. Таблицы, на основе которых строятся представления, называются базовыми.

Представления используются для упрощения и гибкости доступа к таблицам, настройки на восприятие отдельного пользователя. Они также являются механизмом защиты, позволяя ограничить доступ пользователей только к тем данным, которые им нужны.

# Создание представлений

Представление создается при помощи команды

```
create view [владелец .]имя_представления
            [(имя_столбца [, имя_столбца ]...)]
as
    select [distinct] оператор_select
[with check option]
```

- **имя\_представления** - Имя представления. Не может содержать имя базы данных. Не может быть переменной и должно соответствовать правилам для идентификаторов. Чтобы создать представление с тем же именем, что и существующее представление, принадлежащее другому пользователю в текущей базе данных, перед именем представления нужно указать имя владельца. По умолчанию владелец – текущий пользователь.
- **имя\_столбца** - Имена, которые будут использоваться в качестве заголовков столбцов в представлении. Необходимы только в случае, если столбец получен на основе вычисления выражения, выполнения функции или константы, в случае одноименных столбцов при объединении, по желанию пользователя. Заголовки столбцов могут также быть заданы псевдонимами в операторе SELECT. Если заголовки не определены, принимаются имена столбцов подзапроса.
- **select** - Начинает оператор select, определяющий представление.
- **with check option** - Указывает, что операторы не могут изменять данные через представление так, чтобы новые данные не соответствовали условию запроса, определяющего представление. Другими словами, все строки, вставленные и обновленные через представление, должны по-прежнему попадать в это представление.

## Примеры создания:

Создание представления по таблице stores, в котором содержатся сведения о магазинах, находящихся только в Калифорнии (код "CA").

Инструкция with check option проверяет, удовлетворяют ли вставляемые и обновляемые строки условию выбора, определяющему представление. Строки, значение которых в столбце state не равно "CA", не принимаются.

```
create view stores_cal
as
select * from stores where state = "CA"
with check option
```

Создание представления stores\_cal30\_check на основе представления stores\_cal. Новое представление наследует параметр with check option из представления stores\_cal. Кроме того, в его определении также указана инструкция with check option. Поэтому все строки, вставляемые или обновляемые через представление stores\_cal30\_check, должны удовлетворять условиям подзапросов в определениях представлений stores\_cal и stores\_cal30\_check. Строки, в которых значение столбца state не равно "CA" или значение столбца payterms не равно "Net 30", не принимаются.

```
create view stores_cal30_check
as
select * from stores_cal where payterms = 'Net 30'
with check option
```

## Удаление представлений

Команда **DROP VIEW** позволяет удалить представление.

```
DROP VIEW view_name
```

**Пример:**

```
DROP VIEW stores_cal30_check
```

## Использование представлений

- Представления можно использовать в качестве механизма защиты, предоставляя полномочия на доступ к представлениям, а не к таблицам.
- Для переименования представления используется процедура **sp\_rename**.
- Когда над представлением выполняется оператор, Adaptive Server проверяет, существуют ли все объекты базы данных, на которые оно ссылается, допустимы ли они в контексте оператора и не нарушают ли команды обновления правил целостности данных. Если обнаружено хотя бы одно нарушение, выдается сообщение об ошибке. Если проверка заканчивается успешно, команда **create view** “преобразует” запрос к представлению в запрос к базовым таблицам.

## Ограничения на представления

- Представление можно создать только в текущей базе данных.
- Количество столбцов, на которые ссылается представление, не может превышать **1024**.
- Нельзя создать представление на основе временной таблицы.
- Для представления нельзя создать триггеры и индексы.
- Команды **readtext** и **writetext** нельзя выполнять над столбцами представления, имеющими тип **text** или **image**.
- Команда **select**, определяющая представление, не может содержать инструкций **order by** и **compute** и ключевого слова **into**.
- Через представление нельзя вставлять, обновлять и удалять строки, если определяющая его команда **select** содержит оператор **union**.
- Через представление нельзя удалять строки, если определяющая его команда **select** содержит оператор **union**.
- Команду **create view** можно объединять с другими SQL-командами в один пакет.

**Предупреждение:** Если команда **create view** находится внутри блока **if...else** или цикла **while**, Adaptive Server создает схему для представления перед проверкой истинности условия. Это может привести к ошибке, если представление уже существует. Поэтому нужно убедиться в том, что представление с таким именем отсутствует в базе данных.

- В команде **create view** нельзя использовать переменные:

```
declare @p int
select @p = 2
create view v2
as
select * from t1 where c1 > @p
```

**Ошибка:** 7351 – “Local or global variables not allowed in view definition”.

## Решение проблем, связанных с представлениями

- Если структура базовой таблицы представления, определенного командой **select \***, меняется (удаляются или добавляются столбцы), то для того, чтобы в этом представлении были видны новые столбцы, нужно его удалить и создать повторно. Интерпретация символа звездочки и формирование списка столбцов происходит при создании представления.
- Если представление зависит от таблицы или представления, которые были удалены, то при обращении к этому представлению будет выдано сообщение об ошибке. Если создать схему и новую таблицу (представление) с тем же именем, представление снова станет доступно.
- Представление можно переопределить, не изменяя зависимые представления, если после этого переопределения преобразование запросов, определяющих зависимые представления, по-прежнему будет возможным.



## Изменение данных через представления

- Оператор **delete** запрещено использовать с многотабличным представлением.
- Оператор **insert** запрещено использовать в представлении, в которое входят не все столбцы базовой таблицы или представления, не допускающие значения NULL (Adaptive Server не может указать значения для столбцов базовой таблицы или представления, не допускающих значения NULL).
- Строку нельзя вставить через представление, содержащее вычисляемый столбец.
- Оператор **insert** запрещено выполнять над представлением с соединением, созданным с инструкцией **distinct** или **with check option**.
- Оператор **update** можно выполнять над представлением с соединением, созданным с инструкцией **with check option**. Обновление не выполняется, если любой из изменяемых столбцов присутствует в инструкции **where** в выражении, содержащем столбцы из нескольких таблиц.
- Чтобы строку можно было вставить или обновить через представление с соединением, все затрагиваемые этой операцией столбцы должны принадлежать одной и той же базовой таблице. Строку нельзя вставить или обновить в представлении, созданном с инструкцией **distinct**.
- Операторы обновления данных не изменяют вычисляемые столбцы представления и представления с агрегатными значениями.

## Инструкция group by и представления

Если представление создается в целях защиты, нужно соблюдать осторожность при использовании агрегатных функций и инструкции group by. Расширение Transact-SQL позволяет объявлять столбцы, не указанные в инструкции group by. Если указан столбец, отсутствующий в инструкции group by, будут возвращены все строки.

Например, следующий запрос вернет все 18 строк – возможно, больше, чем требовалось:

```
select title_id, type, sum(total_sales)
      from titles
group by type
```

В то время как следующий запрос вернет одну строку для каждого значения столбца **type** (6 строк):

```
select type, sum(total_sales)
      from titles
group by type
```

## Инструкция **distinct** и представления

Инструкция **distinct** определяет представление как объект базы данных, не содержащий повторяющихся строк. Строка считается повторением другой строки, если значения всех столбцов первой строки совпадают со значениями столбцов второй. Значения **NULL** считаются повторяющимися.

Если запрос выбирает подмножество столбцов представления, некоторые строки могут казаться идентичными друг другу. Если некоторые комбинации столбцов, выбираемых запросом, содержат одни и те же значения, то будет казаться, что результирующий набор содержит повторяющиеся строки. Однако строки самого представления останутся уникальными. Adaptive Server применяет инструкцию **distinct** к определению представления при первом обращении к представлению (перед операциями проекции и выборки), поэтому все строки представления являются уникальными.

При определении представления можно несколько раз указать инструкцию **distinct** в составе агрегатных функций или инструкции **group by** оператора **select**, чтобы избавиться от повторяющихся строк. Например:

```
select distinct count(distinct title_id), price
from titles
```

Инструкция **distinct** распространяется только на это представление и не влияет на другие представления, образованные от него.

## Параметр with check option и представления

- Если представление создано с параметром with check option, все строки, вставляемые или обновляемые через это представление, должны удовлетворять его условию отбора.
- Если представление создано с параметром with check option, все представления, которые от него образованы, должны удовлетворять его условию отбора. Все строки, вставленные и обновленные через производное представление, должны оставаться видимыми в этом представлении.

## Получение информации о представлениях

Для отображения информации о таблицах или представлениях, от которых зависит данное представление, и об объектах, которые на него ссылаются, используется процедура **sp\_depends**.

Для отображения текста представления, который хранится в таблице syscomments, выполните процедуру **sp\_helptext**, указав имя представления в качестве параметра.

Использование нескольких ключевых слов distinct и выражения “заголовок\_столбца = имя\_столбца” в операторе select является расширением Transact-SQL.

По умолчанию полномочия на выполнение команды create view принадлежат владельцу базы данных, который может передать их другим пользователям.

Если представление и его базовые объекты принадлежат разным пользователям, то пользователю, вызвавшему представление, должен быть предоставлен прямой доступ к этим объектам. Например, если представление извлекает данные из таблицы, доступ к которой запрещен данному пользователю, выборка завершится ошибкой.

Если представление и объекты принадлежат одному пользователю, применяются специальные правила. Пользователю, который вызывает представление, автоматически назначаются “неявные полномочия” доступа к объектам представления, даже если он не может обращаться к ним напрямую. С помощью представлений можно предоставить ограниченный доступ пользователям к данным таблиц, не предоставляя им прямого доступа к таблицам. Это позволяет использовать представление как механизм защиты. Например, с помощью представления можно предоставить пользователю доступ только к определенным столбцам и строкам таблицы.